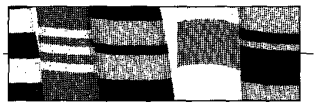


SH3: High Code Density, Low Power

Our SH series microprocessors feature 32-bit RISC architecture with a 16-bit, fixed-length instruction set. We describe SH3, a pipelined implementation of the SH architecture with on-chip cache, MMU, and software-programmable power management. Its higher code density and corresponding improvement in instruction-fetch latency lead to higher performance than typical 32-bit RISC architectures achieve. These features, small die size, and low power consumption make SH3 an ideal microprocessor for portable computing systems or multimedia systems.

Atsushi Hasegawa

Hitachi Microcomputer System, Ltd.

Ikuya Kawasaki
Kouji Yamada
Shinichi Yoshioka

Hitachi, Ltd.

Shumpei Kawasaki
Prasenjit Biswas

Hitachi Micro Systems, Inc.

Over the last several years, the trend in embedded systems has been toward higher processing capability with lower power consumption at lower cost. Various implementations of Hitachi's Super-H (SH) architecture address these requirements. The primary design objectives of this architecture are high performance, small object code size, low power, and small die.

Embedded system designers need more CPU power than ever. In the past, highly integrated embedded systems deployed one or more microprocessors and application-specific acceleration hardware to meet the processing needs of the application domain. To cut costs, however, designers are leaning toward a single, higher performance microprocessor core. This core would replace the required acceleration hardware through software emulation as well as meet the processing requirements of more complex applications. These complex applications include multimedia applications, set-top boxes, nomadic computing systems, sophisticated video games, and so on. The simplicity of RISC architectures and low-cost, low-power implementations make the RISC paradigm more attractive for embedded applications than CISC architectures and implementations of similar performance.[1,2]

For the SH architecture, the RISC paradigm was a natural choice: It avoids the hardware complexity required to decode variable-length instructions as well as the complexity of the control logic for typical CISC instructions.

For embedded systems, it is also essential to examine application code size with particular emphasis on improved instruction-fetch latency. The data space required is usually independent of the instruction set and depends on the application. Increased code density leads to higher processor performance through more effective use of processor memory bandwidth and a lower instruction cache miss rate.[3,4]

Significant advances in VLSI technology and device packaging as well as highly efficient communication technology have created new application areas such as nomadic computing systems. Like personal digital assistants or communication devices, nomadic computing systems operate on batteries. To ensure long battery life, the microprocessor core and associated circuitry must minimize power dissipation.

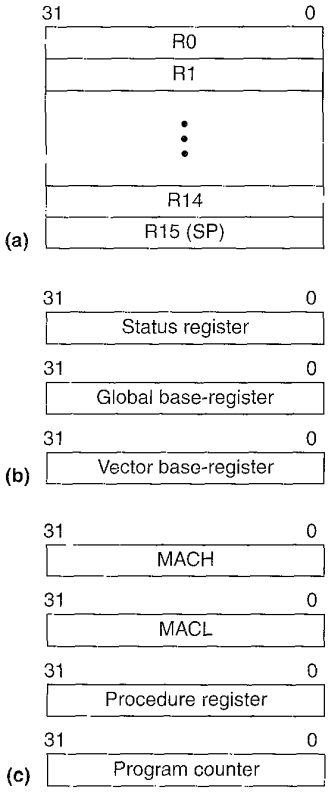
We designed each chip module carefully to reduce total power consumption. Additionally, we provided several system power management features: two low-power modes and the module-stop function, with which programs can stop the clocks of specific peripheral modules.

Instruction set

A typical drawback of 32-bit RISC architectures is a significantly higher code space requirement than that of similar-performance CISC architectures. The SH 16-bit, fixed-length instruction set significantly reduces this drawback and makes the processor particularly attractive for embedded applications. The design team conducted an extensive study to identify the essential and frequently used operations deserving of the restricted operation code space.

Several problems associated with the 16-bit instruction set required careful analysis and corrective compiler optimization strategies. The

0272-1732/95/$04.00 © 1995 IEEE



Bit	Label	Definition
0	T	True/false, carry/borrow, or overflow/underflow bit
1	S	To define MAC instruction result size 0 64-bit result 1 48-bit result on long-word operation, 32-bit result on word operation
4-7	I (3:0)	Interrupt mask bits
8-9	Q, M	Temporal flags for divide operation

(d)

Figure 1. SH user programming model register set: general registers (a), control registers (b), system registers (c), and status register format (d).

short instruction format allows only a limited number of general registers and fewer operands, and less bit immediate data. Short instructions also restrict addressing because they have fewer bits for address offsets. Dropping rarely used instruction set features, careful operation code encoding, and intelligent compilation or coding retained the RISC paradigm advantages, yet resulted in high-density code. Bunda et al.[4] studied a similar strategy starting with Patterson and Hennessy's ideal machine[5], the DLX instruction set.

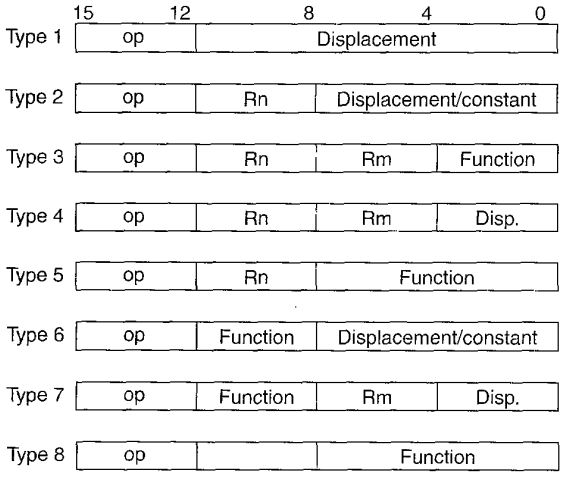


Figure 2. SH instruction formats. Table 1 lists the instructions of each type.

Figure 1 shows the register set of the SH user programming model. Word represents 16 bits, and long-word, 32 bits. There are 16 general registers, relatively few compared to typical RISC architectures. The global base-register functions as a base address register in some addressing modes. The vector base register stores the base address of the exception-processing vector table. On execution of subroutine call instructions, the procedure register stores the return address. Registers MACH and MACL store the results of multiply and multiply-accumulate (MAC) instructions. The status register stores the interruption mask level and the previous instruction results. The T bit stores the result of a conditional check operation, a carry, or overflow information from arithmetic and shift operations. The M and Q bits save intermediate results of divide operations, and the S bit defines the size of the MAC instruction's results.

The SH architecture generally operates using load-store instructions. It uses move and a few other instructions to transfer data from or to memory. Basic SH operations are of two operand types. There are eight instruction types; each instruction's first four bits specify its type.

The architecture supports delayed conditional and unconditional branching. Unconditional branch instructions support 12-bit displacement; conditional branch instructions, 8-bit. The architecture also provides undelayed conditional branch instructions. Since SH has conditional compare instructions and sets results to the T bit, there are only two kinds of conditional branch instructions: branch if true (BT) and branch if false (BF). Figure 2 shows the instruction formats, and Table 1 lists the instructions.

Table 1. Instruction types.

Type	Instructions
1	BRA, BSR
2	MOV, ADD
3	ADD, ADDC, ADDV, SUB, SUBC, SUBV, MUL, MULS, MULU, DMULS, DMULU, MAC, NEG, NEGC, DIV0S, DIV1, MOV, SWAP, XTRCT, EXT, CMP/cond, AND, NOT, OR, TST, XOR
4	MOV
5	MOVT, TAS, DT, ROT, ROTC, SHA, SHL, JMP, JSR, BRAF, BSRF, LDS, LDC, STS, STC
6	CMP/EQ, AND, OR, TST, XOR, MOV, MOVA, BF, BT, BF/S, BT/S, TRAPA
7	MOV
8	CLRT, SETT, CLRMAC, DIV0U, NOP, RTE, RTS, SLEEP

Table 2. Addressing modes.

Addressing mode	Instruction format
Direct register	Rn
Indirect register	@Rn
Postincrement indirect register	@Rn+
Predecrement indirect register	@–Rn
Indirect register with displacement	@(disp:4,Rn)
	@(disp:8,GBR)
Indirect indexed-register	@(R0,Rn)
	@(R0,GBR)
PC-relative	@(disp:8,PC)
Conditional branch	disp:8
Unconditional branch	disp:12
Immediate	#imm:8

```
          SH                          Conventional RISC
          MOV.L  @(const1,PC),R1
          MOV.W  @(const2,PC),R2
          .                           MOVI   #H'5678,R1
          .                           MOVHI  #H'1234,R1
          .                           MOVI   #H'1234,R2
const1:   .dc.L  H'12345678
const2:   .dc.W  H'1234
```

(a)

```
          SH                          Conventional RISC
          MOV.L  @(disp1,PC),Rn
          BRAF   @Rn
here:
          .                           BR     target
          .
          .
disp1:    .dc.L  target - here
```

(b)

Figure 3. Large constants (a) and long-distance branch (b).

Table 2 lists the SH addressing modes. The move instruction has rich addressing modes for long-word load and store, but provides fewer modes for byte and word operands. The displacements for indirect-register-with-displacement and indirect-indexed-register addressing modes are restricted to four bits. In PC-relative addressing mode, the displacements are eight bits. To increase addressing capability, each displacement is scaled by operand size.

To support DSP applications, the instruction set includes a multiply-accumulate instruction. The MAC instruction loads two operands from memory, multiplies those operands, and adds the multiplication result to the 64-bit register pair MACH and MACL. Two general registers point to each operand. MAC instructions can use word or long-word operands.

Immediate fields and address displacements

In spite of its smaller immediate-operand fields, the SH architecture handles constants efficiently. Due to the 16-bit instruction format, the immediate operands for MOV, ADD, and logical operations are assigned eight bits. To set a constant value larger than the assigned eight bits in a general register, SH employs move instructions with PC-relative addressing mode: move word (MOV.W @(disp,PC),Rn) and move long word (MOV.L @(disp,PC),Rn).

In contrast, most conventional RISC architectures have instructions load 16-bit constants, such as MVI and MVHI in the Sparc architecture. MVI assigns a 16-bit constant to a whole register, and MVHI only sets the upper 16 bits of a general register. Most conventional RISCs also have ADD and SUB instructions with 16-bit immediate operands.

Compared to conventional RISCs, SH uses fewer or an equal number of bits of code and fewer or an equal number of instructions to set any constant value to the registers (see Figure 3). To set eight bits, both SH and the conventional RISC use one instruction: SH uses 16 bits, whereas the conventional RISC requires 32 bits. For a 16 bit constant, SH needs one instruction and an additional 16-bit literal, resulting in 32 bits of code. The conventional RISC uses one instruction, 32 bits. For 32-bit constants, SH needs one

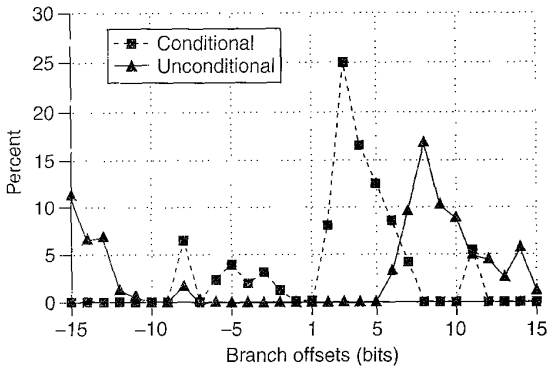
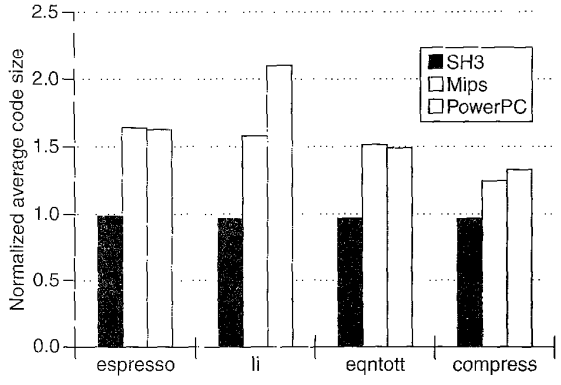


Figure 4. Branch offsets. The branch offsets for conditional branch instructions were small; those for unconditional branches varied widely.



Program	SH3	PowerPC	Mips
espresso	-opt=1	-O	-O2 -G59 -W1 -jmpopt -lfastm
li	-opt=1 -speed -size	-O -qro	-O2 -G59 -lfastm
eqntott	-opt=1 -speed -size	-O	-O2 -O59
compress	-opt=1 -speed -size	-O -qro	-O2 -G32 -lfastm

Figure 6. Object code size of four SPECint92 programs (code sizes normalized with SH equal to 1.0) and compilation options (table).

instruction and a 32-bit literal, 48 bits total, while the conventional RISC needs two instructions, 64 bits.

SH can directly address an array or a record with 16 long-word entries, as the MOV instruction displacement field is four bits. To address a larger record or array, SH requires two instructions: one to compute the effective access address

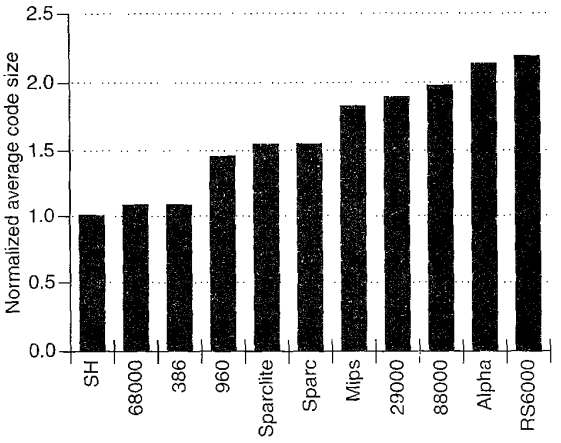


Figure 5. Average object code size for Stanford benchmark suite (code sizes normalized with SH equal to 1.0).

and a second for the actual load or store operation. Conventional RISC architectures use one instruction with a 16 bit displacement field to address a larger record or array.

The displacement in branch instructions is the offset of the branch target from the current program counter. Figure 4 shows our analysis of branch offsets of some embedded programs, such as automobile engine and servomotor control programs. The branch offsets for conditional branch instructions were small; those for unconditional branches varied widely.

Performance analysis

Figure 5 compares the average size of object code for the Stanford benchmark suite on SH and other microprocessors. We compiled the programs on the latest releases of GNU C compilers for each architecture with all optimizations enabled. The average SH object code size is similar to that of CISC architectures considered (the 68000 and i386 series). Compared with that of typical RISC architectures, SH code size is significantly smaller.

Figure 6 shows the ratio of object code sizes for some SPECint92 programs. We compiled the Mips programs on a 5000/R3000 DEC station with an Ultrix C compiler, and used a 320/Power station with an AIX3.2 XL C compiler for the PowerPC programs. We compiled the SH programs with SH/C version 2.0, release 10.

Table 3 shows cache miss ratios for SH and Mips architectures. (Mips values are from Gee et al.[3]) For the SH microprocessor, we used compiling option -opt=1 -speed -size except on the espresso benchmark, which we compiled with the -opt=1 option only. Both measurements used unified, four-way, set-associative cache configurations and 16-byte

Table 3. Cache miss ratios (cache misses/total accesses) at various cache sizes.

	4-Kbyte cache		8-Kbyte cache		16-Kbyte cache		32-Kbyte cache		64-Kbyte cache	
Program	SH3	Mips	SH3	Mips	SH3	Mips	SH3	Mips	SH3	Mips
li	0.0289	0.0430	0.0151	0.0169	0.0105	0.0071	0.0074	0.0035	0.0015	0.0003
compress	0.0234	0.0352	0.0213	0.0323	0.0194	0.0296	0.0174	0.0262	0.0149	0.0218
eqntott	0.0103	0.0114	0.0098	0.0107	0.0095	0.0103	0.0090	0.0096	0.0080	0.0085
espresso	0.0248	0.0295	0.0174	0.0184	0.0118	0.0091	0.0079	0.0043	0.0019	0.0008

lines. The SH architecture has an obvious advantage for cache sizes smaller than 16 Kbytes. With a larger unified cache, the effect of the code density is not significant.

Implementation example

SH7708 and SH7702, the first two microprocessors in the SH3 series, are low-power, high-performance implementations of the SH architecture.[7] Each microprocessor has a 32-bit CPU, memory management unit (MMU), power management unit, flexible bus controller, and various on-chip peripheral modules. SH7702 is the low-cost version of SH7708; it has a 2-Kbyte unified instruction/data cache, whereas SH7708's is 8 Kbytes. SH7702 supports 8- and 16-bit data buses; SH7708 supports these and 32-bit data buses. The smaller bus width allows the SH7702 to fit in a 120-pin TQFP.

Table 4 lists the basic features of these two processors, and Figure 7 shows the combined block diagram.

Low power

Related to power consumption is the technology for heat radiation. Consumed power changes into heat, so microprocessors must radiate it to prevent thermal runaway. A special package such as a ceramic pin-grid array with heat sink can radiate 10 watts or more, but its space expense may be unacceptable for embedded systems. On the other hand, with a 42-alloy lead frame, an ordinary quad-inline-flat package, which is thin, radiates up to 0.7 watts. With a

Table 4. SH3 function outline.

Feature	SH7702	SH7708
Performance	45 MIPS at 45 MHz	60 MIPS at 60 MHz
Power (typical)	450 mW	600 mW
Cache	2 Kbytes, direct mapped	8 Kbytes, 4 way, set associative
TLB	128 entries, 4 way	128 entries, 4 way
Bus interface	ROM/SRAM/DRAM/EDO DRAM/PCMCIA	ROM/SRAM/DRAM/EDO DRAM/PCMCIA/SDRAM
Peripherals	3 timers, serial, RTC	3 timers, serial, RTC
Data bus width	8/16 bits	8/16/32 bits
Process	0.5 µm, 3-metal CMOS	0.5 µm, 3-metal CMOS
Die size	5.7 mm×5.8 mm	6.6 mm×6.6 mm
Package	120-pin TQFP	144-pin QFP

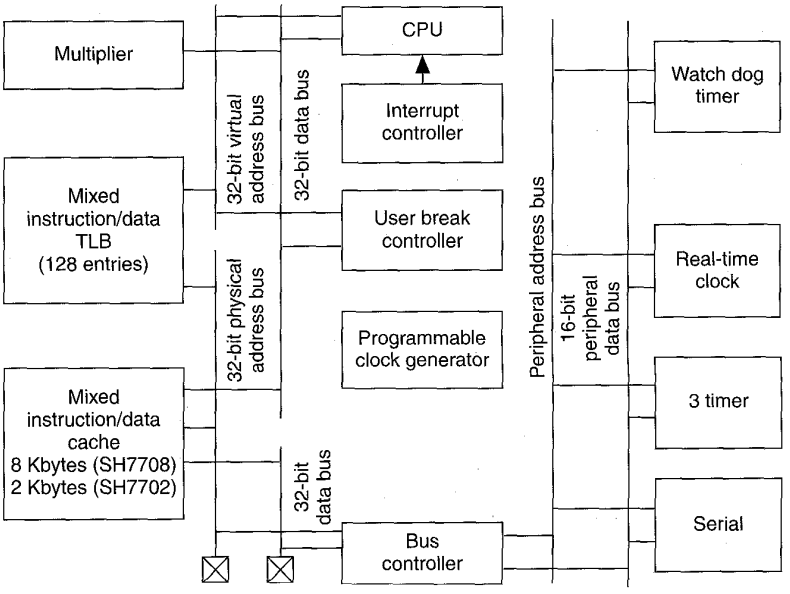


Figure 7. SH3 block diagram.

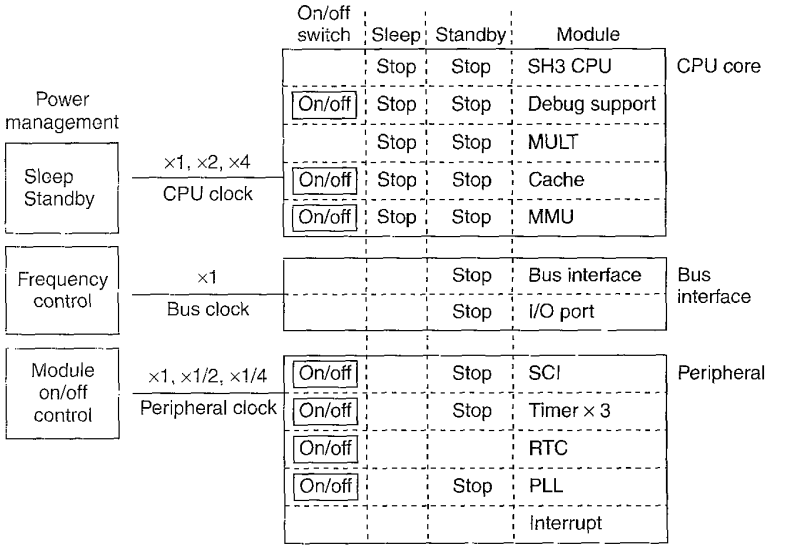


Figure 8. Software-controlled power management.

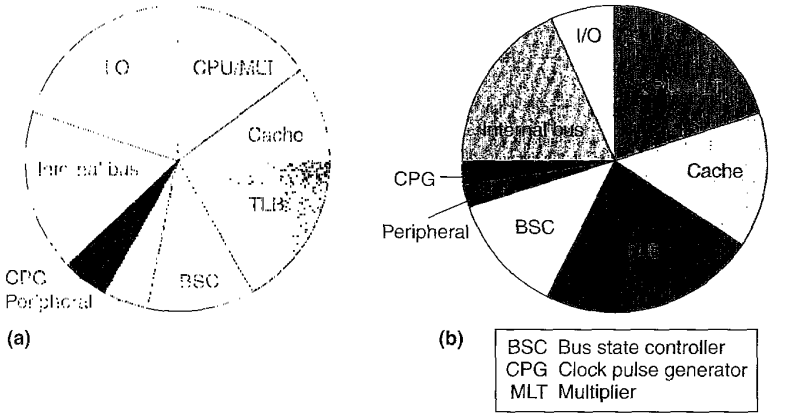


Figure 9. Power dissipation of SH3 modules: CPU clock, 60 MHz; bus clock, 60 MHz; typical total dissipation, 400 mW (a); CPU clock, 60 MHz; bus clock, 15 MHz; typical total dissipation, 290 mW (b).

copper-lead frame it radiates up to 1.2 watts. To satisfy the cost requirement, the power dissipation must fall below any of these figures. SH implementations meet this requirement.

Several features built into the SH3 ensure low-power operation. We classify these features into two categories: software-controlled power management and hardware power-saving features transparent to the software.

Software-controlled power management. The SH3 microprocessors support two power reduction modes, standby and sleep (see Figure 8). In standby mode, all operations but the real-time clock (RTC) stop, and the CPU waits for any external interrupt. If the user does not use the RTC in standby mode, current consumption is less than 1 µA. Otherwise, it remains less than 4 µA.

In sleep mode, the CPU core stops operation, peripheral control modules and the bus controller remain active, and operations such as memory refresh continue. Each peripheral control unit has an on-off switch, allowing the user to selectively disable unused modules.

SH3 has three types of clocks: bus, CPU, and peripheral. For each system, bus clock (used for bus transactions) is fixed, and users choose a specific frequency. The CPU clock runs the CPU, MULT, cache, and MMU. In other words, basic instruction execution depends on the CPU clock. The user can set the CPU clock to the same as, twice, or four times the bus clock frequency. The peripheral clock can be the same as, half, or one-fourth the bus clock frequency. Users can change the CPU and peripheral clock frequencies dynamically by setting the frequency control register.

Microarchitecture for low power. Reducing power dissipation is vital for processors in portable computing systems such as PDAs. Also, to achieve high performance and support general purpose operating systems, it is practically essential to have an on-chip cache and MMU with a translation look-aside buffer (TLB). Because the cache and TLB account for about a third of the total power dissipation, low-power, high-performance processors must keep power dissipation of these modules low. Figure 9 shows the average power dissipation of each SH3 module.

The SH3 architecture uses three schemes to reduce the cache power dissipation. With the first approach, only one

of the four data arrays turns on—the one that address comparison matches to the address in the tag array registers. Figure 10 shows timing relations between the address comparison and the data array operation.

The sequential operation of tag array comparison followed by data array read is only possible when address comparison determines a hit or miss before the data array starts its operation. This depends on the operating frequency. When the hit or miss cannot be determined before the data array operation begins, the four data arrays operate in parallel. The processor automatically switches between sequential and parallel operation depending on operating frequency. Sequential operation is possible at up to 40 MHz. Beyond 40 MHz, the four data arrays operate in parallel. The architecture supporting this mode is called automatic power-save architecture (APSA).

The second power-saving mechanism is the pulse word technique (PWT). In this scheme, a word line is driven in short pulses. The pulse level does not swing fully between V_{CC} and ground level; it is restricted to a narrow range, wide enough for correct sense amplifier operation. The pulse word technique is effective regardless of operating frequency.

The third hardware power-saving feature is the isolated bit line technique (IBLT). Column switches are placed on bit lines between memory cells and latch-type sense amplifiers. The sense amplifiers engage after the column switches turn off. The column switches isolate parasitic capacitors of bit lines from the active sense amplifiers, which decreases the sense amplifier loads.

Figure 11 shows how these three schemes lower cache power dissipation.

The SH3 series TLB is a four-way, set-associative array with 128 entries. In defining the most appropriate TLB structure, the SH3 design team compared a fully associative TLB with 64 entries to the set-associative TLB for area and power dissipation. The two options have almost the same performance (miss ratio).[9] The size and power dissipation of the set-associative TLB, however, is 10 percent less. Since the time window allowed to the TLB is almost the half that of the cache, the APSA-based mechanism would not work for the TLB. Therefore, the team chose another architectural scheme for the TLB to reduce power dissipation.

The processor accesses the TLB on each memory data ref-

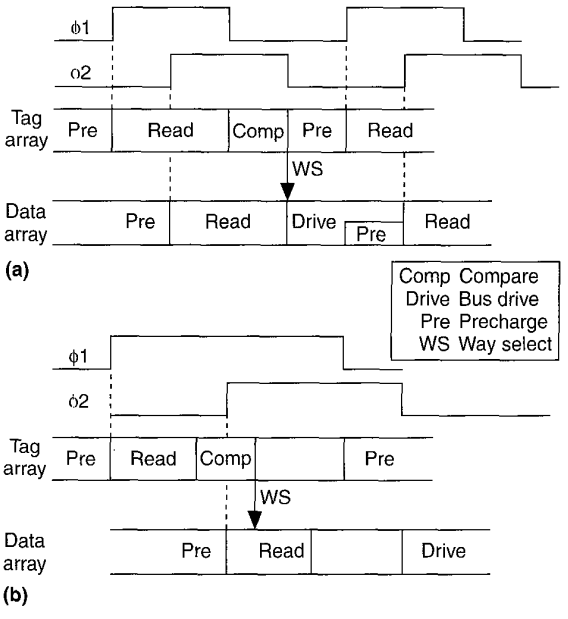


Figure 10. Timing chart of developed cache memory: high frequency (a) and low frequency (b).

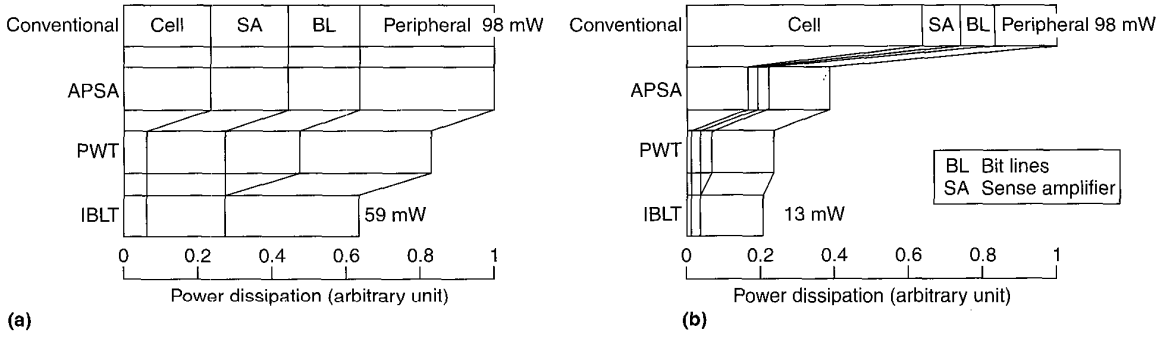


Figure 11. Effects of three power-saving schemes: high frequency, 60 MHz (a); and low frequency, 40 MHz (b).

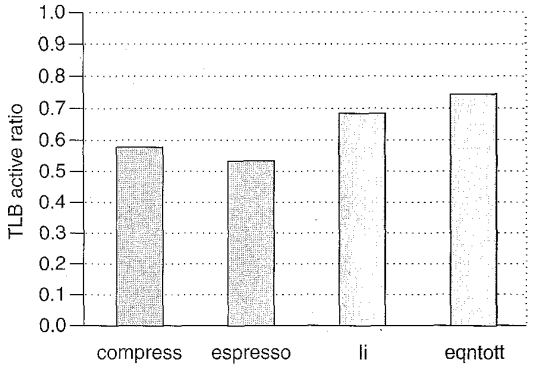


Figure 12. SH3 TLB active ratio in SPECint.

erence. However, during instruction fetch, it checks the TLB only on branch target instruction fetches and on instruction fetches across page boundaries. When the processor does not consult the TLB, a buffer containing the physical address information obtained at the last TLB access provides address translation information. The number of TLB accesses depends entirely on the application program.

Figure 12 shows the effectiveness of this architectural scheme for four SPECint benchmarks. The TLB active ratio represents the total number of times the TLB is activated divided by the total number of memory references for program execution.

The design team also applied power reduction design techniques to other modules. For instance, the clock pulse generator provides the clock to the multiplier module selectively. When the CPU decodes an instruction for the multiplier to execute, it sends a command to the multiplier and signals the clock pulse generator to provide the clock to the multiplier. Otherwise, the multiplier clock remains disabled.

SH7708 typically dissipates 400 mW at 60 MHz operation. A fully static design makes power dissipation directly proportional to clock frequency. Moreover, software-controlled power management further reduces the processor's average power consumption.

Figure 13 is a photograph of the SH7702 die. Die size is 5.83 mm×5.73 mm, and the CPU core area is 4.2 mm^2 in a 0.5-µm, 3-metal CMOS process. It consists of 418,000 transistors, 187,000 of which are used for the cache and TLB.

THE SH ARCHITECTURE USES a 16-bit, fixed-length instruction set. On the average, this instruction set attains better code density than most other architectures. High code density requires less system memory and makes the system less expensive. In addition, it implies a higher cache hit ratio,

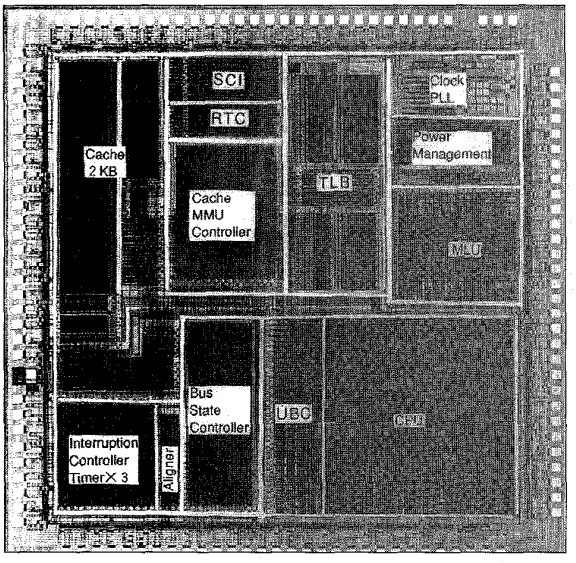


Figure 13. Die photo of SH7702.

which, in turn, results in higher system performance, less bus traffic, and lower power consumption.

We are currently extending the SH architecture to adapt to new market requirements. In one direction, we will add single-precision floating-point instructions for consumer 3D graphics. In another, we will provide complete DSP capability in a single processor engine.

Acknowledgments

We appreciate the efforts of other members of the SH project team at Hitachi, Hitachi Microcomputer System, and Hitachi Microsystems. We thank Yasuhiko Saitou for his assistance in benchmark program compilation and Gautam Dewan for some of the simulation results.

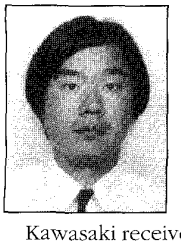
References

1. D.A. Patterson and J.L. Hennessy, *Computer Architecture: A Quantitative Approach,* Morgan Kaufmann Publishers, San Francisco, Calif., 1990.
2. G. Radin, "The 801 Minicomputer," *Proc. Symp. Architectural Supports for Programming Languages and Operating Systems,* IEEE Computer Society Press, Los Alamitos, Calif., 1982, pp. 39-47.
3. P. Steenkiste, "The Impact of Code Density on Instruction Cache Performance," *Proc. Int'l Symp. Computer Architecture,* IEEE Computer Society Press, Los Alamitos, Calif., 1989, pp. 252-259.

4. J. Bunda et al., "16-Bit vs. 32-Bit Instructions for Pipelined Microprocessors," *Proc. Int'l Symp. Computer Architecture,* IEEE CS Press, 1992, pp. 237-246.
5. J.D. Gee et al., "Cache Performance of the SPEC92 Benchmark Suite," *IEEE Micro,* Vol. 13, No. 4, Aug. 1993, pp. 17-27.
6. R. Uhlig et al., "Design Tradeoffs for Software-Managed TLBs," *ACM Trans. Computer Systems,* Vol. 12, No. 3, Aug. 1994, pp. 175-205.
7. S. Narita et al., "A Low-Power Single-Chip Microprocessor with Multiple Page-Size MMU for Nomadic Computing," *Proc. Symp. VLSI Circuits,* IEEE, 1995, pp. 59-60.

Atsushi Hasegawa is a senior engineer in the Microcomputer System Engineering Department at Hitachi Microcomputer System, Ltd., where he currently designs microprocessors. His research interests are microprocessor architecture, performance measurements, and parallel systems.

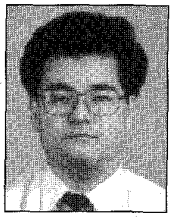
Hasegawa received his BS degree in computer science from the University of Electro-Communications in Japan. He is a member of the IEEE Computer Society and the ACM.



Ikuya Kawasaki is a section manager in the 32-Bit Microcomputer Development Office of the Semiconductor and Integrated Circuits Division of Hitachi, Ltd. He designs 32-bit microcomputers, and his research interests are microprocessor architecture and VLSI design methodology.

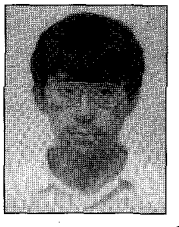
Kawasaki received a BS in electrical engineering from the University of Tokyo and an MS from the University of Washington.

Kouji Yamada is a member of the Microprocessor Development Group at Hitachi's Semiconductor Development Center. He currently works on design and performance analysis of the SH processor and optimizing compiler. Previously, he researched the compiler and designed the PA-RISC processor at Hitachi's Systems Development Lab. His interests include multimedia system and application software.



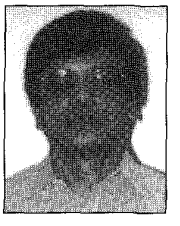
Shinichi Yoshioka is an engineer in the 32-bit Microcomputer Development Office in Hitachi, Ltd.'s Semiconductor and Integrated Circuits Division. He designs 32-bit microcomputers and is interested in microarchitecture in general, especially in intermediate fields between hardware and software.

Yoshioka received his BE degree in applied physics from the University of Tokyo.



Shumpei Kawasaki is a VLSI design engineer at Hitachi Microsystems, in San Jose, California. His current research and development interests are in computer and instruction set architecture.

Kawasaki received a BA in mathematics from Knox College and an MS in computer science from the University of Illinois at Urbana-Champaign.



Prasenjit Biswas is the group manager for processor architecture and performance analysis at Hitachi Microsystems, San Jose, California. Prior to joining Hitachi, he worked for Cyrix Corporation as a senior computer architect, for Texas Instruments' Computer Science Lab, and as an assistant professor of CSE at SMU, Dallas.

Biswas' current research and development interests are in computer architecture, parallel processing, and performance modeling. He has published more than thirty refereed articles. Biswas is a member of the IEEE and the ACM.

Direct questions concerning this article to Atsushi Hasegawa at Microcomputer System Engineering Department, Hitachi Microcomputer System, Ltd., 5-22-1 Jousui Hon-cho, Kodaira, Tokyo 187 Japan; hasegawa@hitachi-mc.co.jp.